This fallback structure grants a lot of flexibility. But for it to provide the defaults you need, you have to know how to configure them. They're configured in the file reference.conf and placed in the root of the JAR file; the idea is that every library contains its own defaults. The configuration library finds all the reference.conf files and integrates these settings into the configuration fallback structure. This way, all the needed properties of a library always have a default, and the principle of always getting a value back is preserved. (Later, you see that you can explicitly stipulate defaults programmatically, as well.)

You already saw that the configuration library supports multiple formats: Java properties, JSON, and HOCON. Nothing is stopping you from using multiple formats in a single application. Each file can be used as the fallback of another file. And it supports the possibility of overruling properties with system properties, which is the higher-ranking configuration. The structure is always the same, so the relationships between defaults and overrides are always the same. Figure 7.3 shows the files the config library uses to build the complete tree in priority order.

Most applications use only one of these application file types. But if you want to provide a set of application defaults and then override some of them, as with the example JDBC settings, you can do so. When following this guide, keep in mind that the upper configuration files shown in figure 7.3 will overrule values defined in the lower files.